DESARROLLO DE APLICACIONES II

Matriz de criterios de seguridad

Grupo 9 · Core

20 de octubre de 2026

Contenido

Cómo se compuso la matriz 3
La matriz 4
Criterios en detalle 4
Verificación independiente con Fable 5.1 9
Hallazgos abiertos 10
Carga en Jira 10
Cómo reproducir la verificación 10
Anexo · Capturas de los tests 11
Supuestos 13

Desarrollo de Aplicaciones II · Clase 11 · Taller de seguridad de aplicaciones

Dominio: **Core**, el pasamanos de eventos de la plataforma municipal distribuida. Recibe los eventos asincrónicos de los 9 módulos, valida que el sobre esté bien formado, guarda evidencia y los entrega a quien esté suscripto.

> **Resultado: 7 de 10 criterios cumplen. 3 hallazgos abiertos**, todos verificados ejecutando los tests contra la aplicación real, no sobre el papel. Los tres tienen su ticket y están al final del documento.

Cómo se compuso la matriz

La consigna pide ≥6 web/API, ≥2 de integración y ≥2 mobile; **sin app, 2 de datos sensibles**. El Core no tiene aplicación mobile —es un backend más un dashboard web—, así que se tomó la alternativa prevista:

Categoría	Cantidad	Criterios
Web / API	6	SEC-01 a SEC-06
Integración (mensajería AMQP)	2	SEC-07, SEC-08
Datos sensibles (reemplaza mobile)	2	SEC-09, SEC-10

Los dos de integración son sobre mensajería y no SOAP, porque la integración entre los 9 módulos es por RabbitMQ: los módulos publican en el exchange `muni.inbox` y consumen de su cola `q.<módulo>`.

La matriz

ID	Criterio	Categoría	Referencia	Veredicto
SEC-01	Un módulo no accede al tráfico en el que no participa	Web/API	API1:2023 · ASVS V4.1.3	**Cumple**
SEC-02	Las funciones de administración rechazan a un módulo común	Web/API	API5:2023 · ASVS V4.1.3	**Cumple**
SEC-03	Sin credencial válida no se accede; la firma RS256 frena la manipulación	Web/API	API2:2023 · ASVS V3.5.2	**Cumple**
SEC-04	La cuenta se bloquea tras 5 intentos fallidos	Web/API	API2:2023 · ASVS V2.2.1	**Cumple**
SEC-05	El login no revela si un email existe	Web/API	A07:2021 · ASVS V2.2.3	**Cumple**
SEC-06	El login limita la tasa de intentos por origen	Web/API	API4:2023 · ASVS V11.2.1	**No cumple**
SEC-07	El sobre rechaza campos no previstos y fechas ambiguas	Integración	API6:2023 · ASVS V5.1.2	**Cumple**
SEC-08	Sólo el módulo dueño puede escribir el JSON Schema de su evento	Integración	API5:2023 · ASVS V4.2.1	**No cumple**
SEC-09	El contenido de un evento sólo lo ven origen, destinatarios y el admin	Datos sensibles	API1:2023 · ASVS V8.1.1	**Cumple**
SEC-10	Los datos sensibles no quedan en claro ni en la base ni en los logs	Datos sensibles	API8:2023 · ASVS V7.1.1	**No cumple**

Criterios en detalle

SEC-01 · Un módulo no accede al tráfico en el que no participa

Dado un módulo autenticado que no es ni origen ni destinatario de un evento **Cuando** consulta `GET /api/v1/events`, o fuerza el filtro `?sourceModule=<ajeno>` **Entonces** ese evento no aparece en el listado, y `GET /dashboard/modules/<ajeno>` devuelve 403

Evidencia

```
'habilitaciones' no es ni origen ni destinatario del evento
d1a9a5a6-73de-4bc5-b381-4994f9ef363a, publicado por 'atencion-ciudadana'

GET /events (token de habilitaciones)        -> HTTP 200, 3 resultados
  el evento privado aparece en el listado:   False
GET /events?sourceModule=atencion-ciudadana  -> 0 eventos privados filtrados
GET /dashboard/modules/atencion-ciudadana    -> HTTP 403
```

Nota sobre cómo se probó. El primer intento de test estaba mal planteado: daba por fuga que `rentas` viera eventos de `atencion-ciudadana`. No lo es — `rentas` está suscripto a `ticketCreated` y esos eventos **le fueron entregados**, así que verlos es el comportamiento correcto. El aislamiento sólo se puede probar con un módulo que no sea ni origen ni destinatario, y por eso el test busca ese caso en los datos antes de afirmar nada.

El filtro vive en `EventLogRepository._scoped()` y se aplica del lado del servidor a partir del token, no de los query params: un módulo común queda limitado a lo suyo mande lo que mande.

SEC-02 · Las funciones de administración rechazan a un módulo común

> **Dado** un token de módulo sin `isAdmin` **Cuando** llama a `/dashboard/global`, `/modules/{n}/rotate-secret`, `/dlq/retry-bulk` o `/topology/apply` **Entonces** recibe 403, y el administrador recibe 200 en el control positivo

Evidencia

```
GET   /dashboard/global            (rentas) -> HTTP 403
POST  /modules/obras/rotate-secret (rentas) -> HTTP 403
POST  /dlq/retry-bulk              (rentas) -> HTTP 403
POST  /topology/apply              (rentas) -> HTTP 403
GET   /dashboard/global            (admin)  -> HTTP 200   (control positivo)
```

El control positivo importa: sin él, un 403 a todo también pasaría el test.

SEC-03 · Sin credencial válida no se accede; la firma RS256 frena la manipulación

> **Dado** un cliente sin token, con un token inválido, con un token firmado por otra clave RSA, o con el payload alterado a `isAdmin: true` **Cuando** llama a un endpoint protegido **Entonces** recibe 401 en los cuatro casos

Evidencia

```
sin header Authorization          -> HTTP 401
token con formato invalido        -> HTTP 401
token firmado con otra clave RSA  -> HTTP 401   (intentaba isAdmin=true)
payload alterado a isAdmin=true   -> HTTP 401   (la firma RS256 ya no valida)
```

Los dos últimos son los que importan: se generó una clave RSA nueva y se firmó un token que se declaraba administrador, y se tomó un token legítimo y se le reescribió el claim `isAdmin`. Los dos se rechazan.

Observación que no invalida el criterio pero conviene registrar. `isAdmin` se lee del claim del token, no de la base en cada request. Si a un módulo se le quita el privilegio, su token vigente lo conserva hasta

vencer. La ventana es de 15 minutos (`access_token_ttl_minutes`), que para este sistema es aceptable, pero es una decisión, no un descuido.

SEC-04 · La cuenta se bloquea tras 5 intentos fallidos

> **Dada** una cuenta válida **Cuando** se hacen 5 intentos de login con la contraseña incorrecta **Entonces** la cuenta queda desactivada, y el intento siguiente **con la contraseña correcta** devuelve 403 `ACCOUNT_INACTIVE`

Evidencia

```
intento fallido 1/5 -> HTTP 401 INVALID_CREDENTIALS
intento fallido 2/5 -> HTTP 401 INVALID_CREDENTIALS
intento fallido 3/5 -> HTTP 401 INVALID_CREDENTIALS
intento fallido 4/5 -> HTTP 401 INVALID_CREDENTIALS
intento fallido 5/5 -> HTTP 401 INVALID_CREDENTIALS
ahora con la contrasena CORRECTA -> HTTP 403 ACCOUNT_INACTIVE
   La cuenta esta desactivada. Pedile a alguien de tu equipo que la reactive.
```

Probar el sexto intento con la contraseña **correcta** es lo que prueba el bloqueo. Con una contraseña incorrecta la respuesta sería 401 igual que antes y el test pasaría sin demostrar nada.

Este control estuvo roto y no se notaba: el contador de intentos se incrementaba dentro de la misma transacción que después hacía rollback al lanzar la excepción, así que nunca llegaba a 5. Se arregló con `UserRepository.record_failed_login()` , que confirma la transacción aparte.

SEC-05 · El login no revela si un email existe

> **Dado** un intento de login con un email inexistente y otro con un email real y contraseña incorrecta **Cuando** se comparan las dos respuestas **Entonces** coinciden en status, `code` y `message`

Evidencia

```
email inexistente       -> HTTP 401  code=INVALID_CREDENTIALS  "Credenciales invalidas."
email real, clave mala  -> HTTP 401  code=INVALID_CREDENTIALS  "Credenciales invalidas."
respuestas indistinguibles: True
```

Se ignora el `traceId` , que es distinto en cada request a propósito. (El primer test lo comparaba y daba un falso negativo.)

SEC-06 · El login limita la tasa de intentos por origen — NO CUMPLE

> **Dado** un atacante con acceso de red al Core **Cuando** envía 40 intentos de login seguidos **Entonces** debería recibir HTTP 429 a partir de cierto umbral

Evidencia

```
40 intentos de login seguidos contra un email inexistente
  respuestas: [401]      HTTP 429 recibidos: 0
  no hay middleware de rate limiting en la app
```

Por qué el bloqueo de cuenta (SEC-04) no alcanza. El bloqueo protege a una cuenta, no al servicio, y hay que decirlo con precisión: *sí* frena la fuerza bruta contra una contraseña. Lo que no frena es el resto.

La verificación independiente encontró algo que la primera corrida no había mirado — **una vez bloqueada la cuenta, los intentos siguientes se siguen procesando igual**:

```
intento  4  HTTP 401 INVALID_CREDENTIALS     189 ms
intento  5  HTTP 401 INVALID_CREDENTIALS     190 ms   <- se bloquea la cuenta
intento  6  HTTP 401 INVALID_CREDENTIALS     189 ms   <- ya bloqueada, igual responde
intento 12  HTTP 401 INVALID_CREDENTIALS     189 ms
```

En `auth_service.login()` el chequeo de `user.active` está **después** de `verify_password()`, así que cada intento sobre una cuenta ya bloqueada paga de todos modos el hash bcrypt. El bloqueo no reduce el trabajo del servidor.

Riesgos que quedan en pie:

- **Bloqueo masivo de cuentas.** Los emails son predecibles (`<modulo>@muni.uade.edu.ar`): con 5 requests por equipo se dejan los 9 módulos afuera del dashboard.
- **Password spraying.** 1 a 4 intentos por cuenta contra muchas cuentas nunca dispara el bloqueo.
- **Agotamiento de CPU.** Cada request fuerza un bcrypt de ~190 ms, caro a propósito. Sin límite de tasa, eso es el ataque.

Lo que **no** es este hallazgo: robo de contraseña por fuerza bruta ilimitada. Para eso el bloqueo sí sirve.

SEC-07 · El sobre rechaza campos no previstos y fechas ambiguas

> **Dado** un productor que publica un evento **Cuando** el sobre trae campos no declarados, o un `occurredAt` sin zona horaria **Entonces** el Core responde 422 y no lo procesa

Evidencia

```
sobre bien formado                    -> HTTP 202
sobre + campos no previstos (isAdmin)  -> HTTP 422
occurredAt sin zona horaria            -> HTTP 422
```

El sobre declara `extra="forbid"`. Un campo que el Core no espera no se ignora en silencio: se rechaza. Eso cierra la puerta a que un productor cuele atributos con la esperanza de que alguien los interprete más adelante.

SEC-08 · Sólo el módulo dueño puede escribir el JSON Schema de su evento — NO CUMPLE

> **Dado** el token del módulo `rentas` **Cuando** hace `PUT /api/v1/event-types/workOrderScheduled/schema`, un tipo cuyo `ownerModule` es `obras` **Entonces** debería recibir 403

Evidencia

```
'workOrderScheduled' pertenece a: obras
PUT /event-types/workOrderScheduled/schema  (token de rentas) -> HTTP 200
  schema quedo como: {"type": "object", "required": ["inyectadoPorRentas"]}
  -> rentas reescribio el contrato de un evento de obras
```

El impacto está probado, no supuesto. La verificación independiente siguió el ataque un paso más: después de envenenar el schema, hizo que Obras publicara su propio evento legítimo.

```
OBRAS publica su evento legitimo -> HTTP 422
status: REJECTED   rejectionCode: SCHEMA_VIOLATION
detalles: campoInventadoPorRentas requerido y faltante;
          additionalProperties: 'scheduledFor','workOrderId' no permitidos
entradas en DLQ visibles para obras: 1
  DLQ: workOrderScheduled | SCHEMA_VIOLATION
```

O sea: **todo el tráfico legítimo de Obras de ese tipo se va a la DLQ** hasta que alguien note el schema alterado. Es una negación de servicio contra otro equipo, desde una cuenta común de cualquier módulo, y la víctima no tiene cómo enterarse salvo mirando su DLQ.

No hay pérdida permanente —el Core guarda la evidencia y permite reprocesar—, pero la integración queda interrumpida.

`RegistryService.set_schema()` no recibe el módulo que llama, así que no puede compararlo con `event_type.owner_module`. El endpoint pide autenticación pero no autorización.

SEC-09 · El contenido de un evento sólo lo ven origen, destinatarios y el admin

> **Dado** un evento cuyo `data` trae información de un ciudadano **Cuando** lo pide un módulo que no es ni origen ni destinatario **Entonces** recibe 404 y no ve el payload

Evidencia

```
evento publicado por 'atencion-ciudadana', en el que 'habilitaciones' no participa
GET /events/{id}  (token de habilitaciones) -> HTTP 404
GET /events/{id}  (admin)                   -> HTTP 200   (control positivo)
  el admin si ve el data: {"ticketId": "TK-DEMO-001", "category": "INFRASTRUCTURE"}
```

Responde **404 y no 403**, que es lo correcto acá: un 403 confirmaría que el evento existe.

SEC-10 · Los datos sensibles no quedan en claro ni en la base ni en los logs — NO CUMPLE

> **Dado** un mensaje malformado que contiene datos personales **Cuando** el Core lo rechaza **Entonces** no debería escribir su cuerpo en el log

Evidencia

```
modules.secret_hash de 'obras': f0609c435dc63902...  (64 chars, hex=True)   OK
GET /modules (admin): el secreto NO viaja en la respuesta: True             OK

app/services/event_hub_service.py:371
  logger.warning("malformed_message", queue=queue, preview=raw_body[:200])  FALLA
```

Las dos primeras partes del criterio se cumplen: el secreto de máquina se guarda como hash SHA-256 irreversible y no sale por ninguna respuesta de la API.

La tercera no. Cuando llega un mensaje que no es JSON válido o al que le falta el sobre, el Core escribe sus primeros 200 caracteres en el log de la aplicación. Si ese mensaje traía el DNI o el domicilio de un ciudadano —que es exactamente lo que viaja en el `data` de estos eventos—, ese dato queda en un log que se rota, se archiva y se indexa, y que tiene una audiencia bastante más amplia que la base.

Evidencia de que se ejecuta de verdad. Se provocó un mensaje malformado con un DNI ficticio y se capturó la salida del logger:

```
[warning] malformed_message  module=core  queue=core.inbox
  preview='{"eventType":"citizenRegistered","sourceModule":"ciudadanos",
            "data":{"nombre":"DNI 37.428.915 — Juana Perez — Av. Siempreviva 742"}'

¿'37.428.915' aparece en el log? -> True
```

En producción el logger renderiza JSON, así que ese dato va tal cual al agregador de logs, que tiene una audiencia bastante más amplia que la base.

Qué corregir y qué no. El `preview` del log hay que sacarlo: no agrega nada. El cuerpo completo **sí** se sigue guardando en `dead_letters.raw_body` (hasta 20.000 caracteres), y eso es necesario para poder reprocesar el mensaje; está bajo control de acceso y es una decisión de diseño razonable. Lo que falta ahí no es borrarlo sino **una política de retención**: hoy ese dato personal queda en la base indefinidamente.

Verificación independiente con Fable 5.1

La consigna pide verificar al menos 3 criterios con Fable 5.1. Se eligieron **los tres hallazgos** —no tres que cumplieran— y se le pidió al modelo que escribiera y ejecutara **sus propios tests**, con la instrucción explícita de refutar lo que no se sostuviera: *«un hallazgo que no se sostiene hay que bajarlo; refutarlo vale tanto como confirmarlo»*.

Trabajó sobre su propia copia de la base, en proceso, sin tocar el proyecto.

Hallazgo	Veredicto	OWASP principal	Severidad
SEC-06 · sin rate limiting en el login	**Confirmado**	API4:2023	Media
SEC-08 · sobrescribir el schema ajeno	**Confirmado**	API5:2023	Alta
SEC-10 · PII cruda en el log	**Confirmado**	API8:2023 · ASVS V7.1.1	Media

Los tres se sostuvieron, y la verificación **cambió el informe en tres puntos**:

1. SEC-06 bajó de Alta a Media, y se precisó el riesgo. El planteo original daba a entender fuerza bruta ilimitada contra una contraseña, y eso no es cierto: el bloqueo de cuenta sí la frena. Lo que queda en pie es bloqueo masivo de cuentas, password spraying y agotamiento de CPU. El informe corregido lo dice así.

2. SEC-06 ganó una evidencia que no teníamos. Que una cuenta ya bloqueada siga pagando el bcrypt en cada intento posterior no estaba en la primera corrida. Se verificó aparte, midiendo tiempos, y se confirmó.

3. SEC-08 pasó de impacto afirmado a impacto probado. El primer test mostraba que el `PUT` devolvía 200. El segundo hizo que Obras publicara después su evento legítimo y capturó el 422, el `SCHEMA_VIOLATION` y la entrada en la DLQ. Eso es lo que convierte «se puede editar un campo ajeno» en «se le puede cortar la integración a otro equipo».

También corrigió la referencia de SEC-10: `API8:2023 Security Misconfiguration` encaja mejor que `API3:2023`, porque la falla es de configuración de logging y no de exposición de datos en una respuesta de la API.

Hallazgos abiertos

ID	Hallazgo	Severidad	Arreglo
SEC-06	Sin límite de tasa en `/auth/login`	Media	Middleware de rate limiting por IP sobre los endpoints de autenticación
SEC-08	Cualquier módulo puede reescribir el JSON Schema de un evento ajeno	Alta	Pasar el módulo que llama a `set_schema()` y comparar contra `owner_module`
SEC-10	El cuerpo crudo de un mensaje malformado se escribe en el log	Media	Quitar `preview` del `logger.warning`; el cuerpo ya queda en `dead_letters.raw_body`

SEC-08 es el más urgente, y es el único de severidad alta: se explota desde una cuenta común de cualquier equipo, no necesita condiciones especiales, y el daño le cae a otro módulo que no tiene forma de enterarse salvo mirando su DLQ. El arreglo son tres líneas —pasar el módulo que llama a `set_schema()` y compararlo con `owner_module`, igual que ya hacen las suscripciones y las publicaciones—. El endpoint de schema es la excepción en todo el registry.

Carga en Jira

Los 10 criterios están en `criterios-jira.csv`, listo para importar (*Issues → Import issues from CSV*). Cada fila trae:

- **Summary** con el ID `SEC-xx`
- **Labels**: `seguridad`
- **Issue Type**: `Task` para los que cumplen, `Bug` para los tres hallazgos
- **Description** con el Dado/Cuando/Entonces, la referencia OWASP/ASVS y el veredicto

La columna **Assignee** quedó vacía a propósito: Jira la resuelve contra los usuarios reales del proyecto y se completa en el momento de importar.

Cómo reproducir la verificación

```
cd backend
cp muni_core.db /tmp/pruebas.db        # la bateria bloquea una cuenta: copia descartable
.venv/bin/python ../docs/seguridad/verificar_seguridad.py
```

El script (`verificar_seguridad.py`) corre en proceso con `httpx.ASGITransport` contra la aplicación real, sin levantar un servidor. Imprime el veredicto y la evidencia cruda de cada criterio y los deja en `resultados.json`. La salida completa de la corrida está en `evidencia-tests.txt`.

Tres cosas que el script hace a propósito:

- **Trabaja sobre una copia de la base.** SEC-04 desactiva una cuenta y SEC-08 reescribe un schema. Correrlo contra la base de desarrollo la dejaría sucia.
- **SEC-04 va último**, porque bloquea la cuenta de `ambiente`.
- **Incluye controles positivos.** En SEC-02 y SEC-09 se verifica también que el administrador sí pueda. Sin eso, un servicio que respondiera 403 a todo pasaría los tests.

Anexo · Capturas de los tests

La corrida completa en texto está en `evidencia-tests.txt`; acá van las capturas de los cuatro criterios que más importan y el resumen final.

Resumen de la corrida

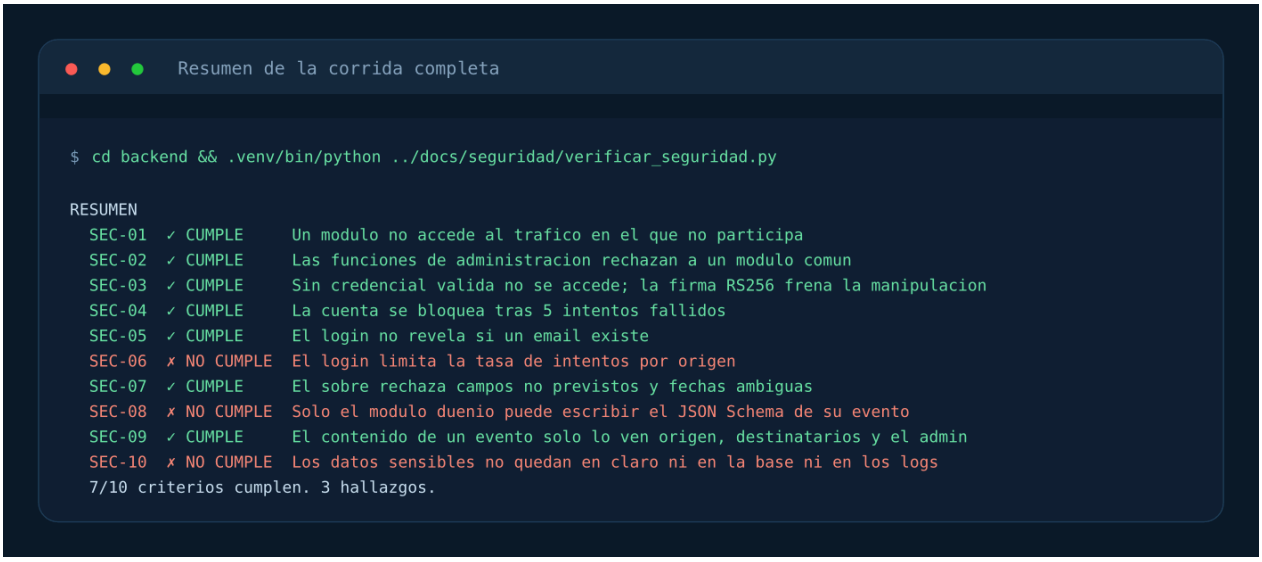


Figura 1: Resumen de la corrida: 7 de 10 criterios cumplen

SEC-04 · El bloqueo de cuenta funciona

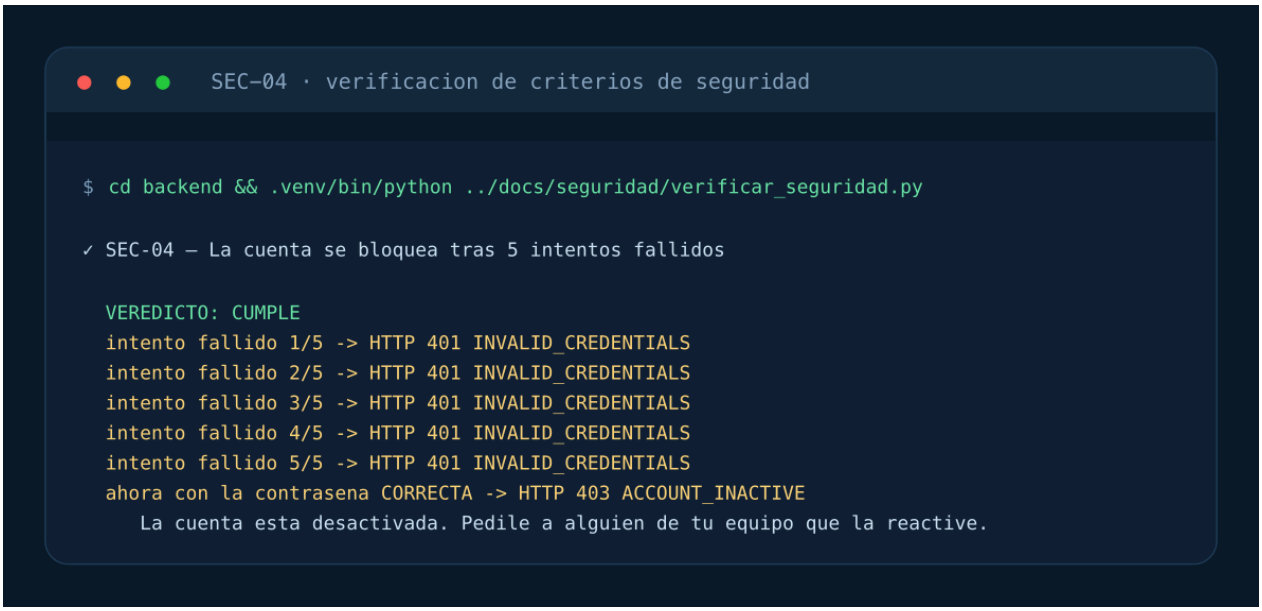


Figura 2: Cinco intentos fallidos y la cuenta queda desactivada

Lo que prueba el bloqueo es el sexto intento, hecho **con la contraseña correcta**: devuelve 403 `ACCOUNT_INACTIVE`.

SEC-06 · Sin límite de tasa en el login

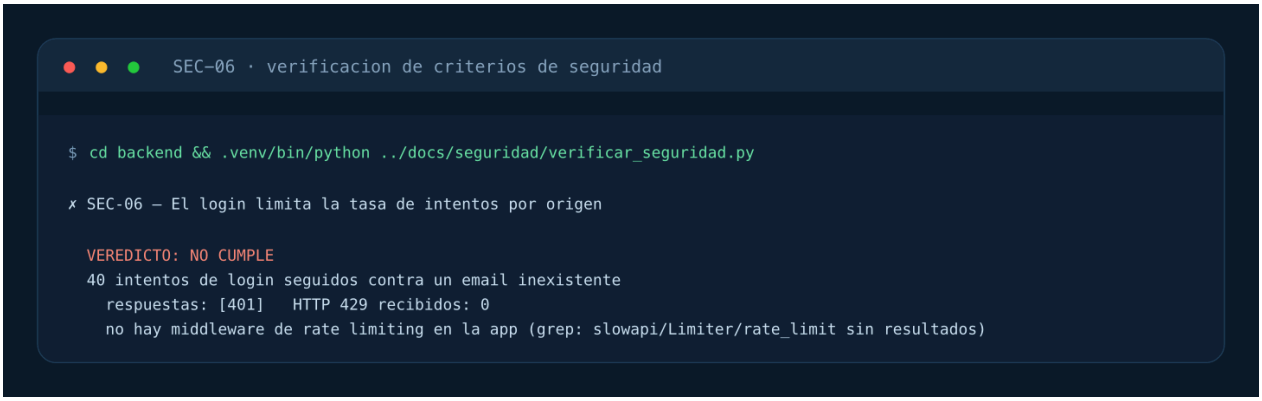


Figura 3: Cuarenta intentos seguidos, ningun 429

SEC-08 · Un módulo reescribe el schema de otro

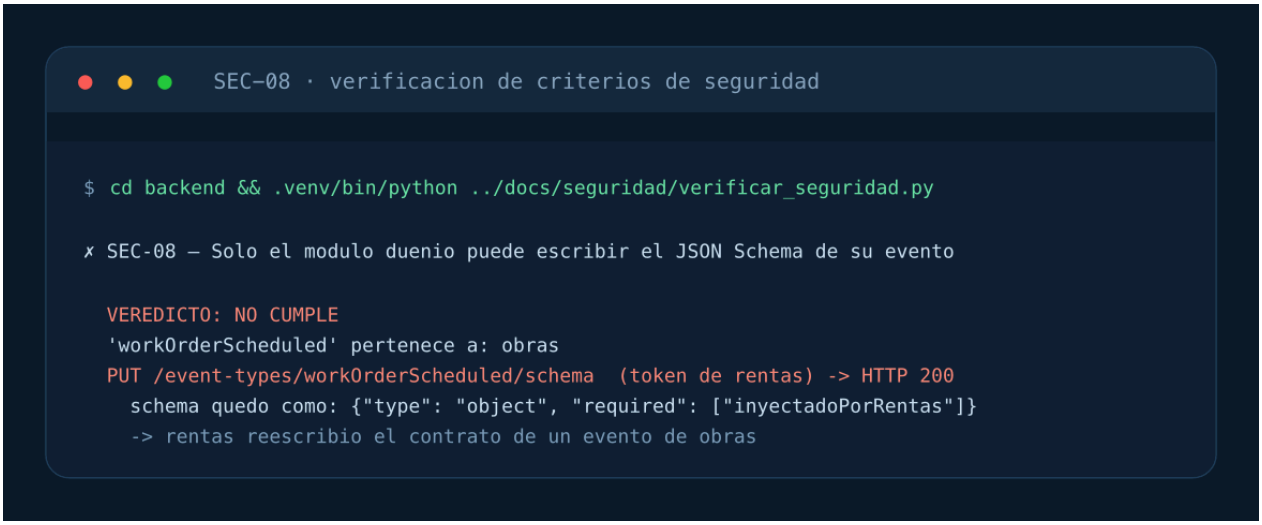


Figura 4: Rentas reescribe el schema de un evento de Obras y recibe HTTP 200

Acá el `HTTP 200` es la falla: debería haber sido un 403.

SEC-10 · Datos personales en los logs

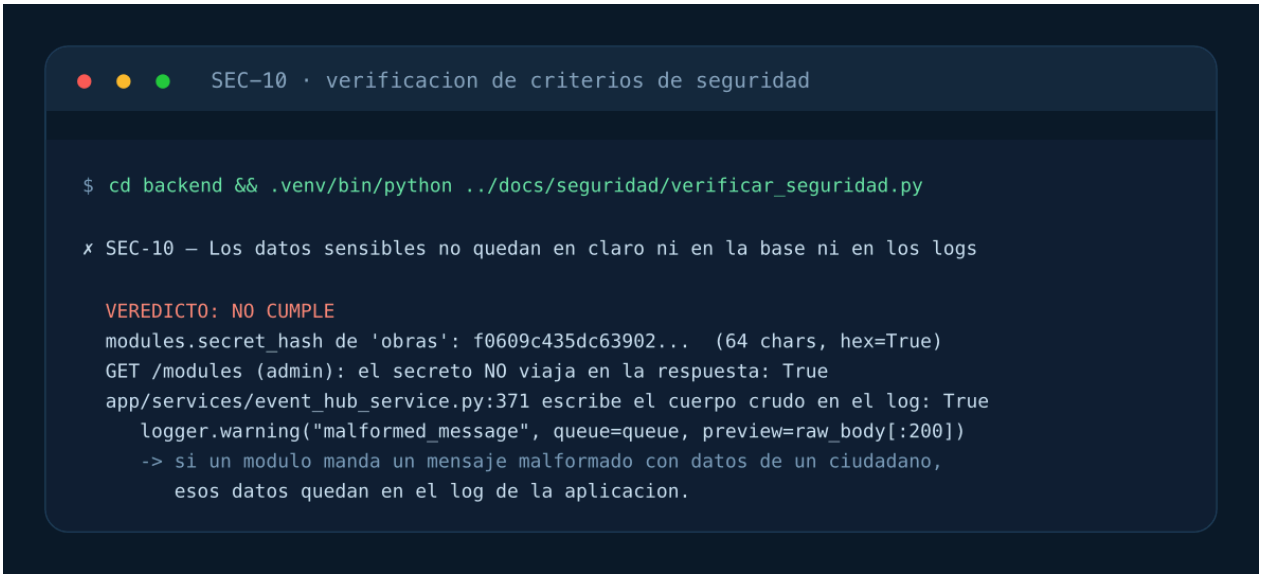


Figura 5: El cuerpo crudo del mensaje malformado queda en el log

Supuestos

- **Grupo 9 / dominio Core.** El Core es el módulo 9 de la plataforma y figura como «Equipo 9» en el catálogo, así que el entregable se nombró `Clase11_Grupo9_Core_Seguridad.pdf`. Si la cátedra numera distinto, se cambia el nombre del archivo y nada más.
- **Los 2 criterios de datos sensibles reemplazan a los 2 de mobile**, por la alternativa que da la consigna para los equipos sin app.
- Los criterios se verificaron contra el **entorno de desarrollo**, que es el único que existe hoy: todavía no hay deploy de Test ni de Producción. Dos de los controles (`JWT_PRIVATE_KEY` obligatoria y `CORS_ORIGINS` acotado) sólo se pueden verificar de verdad cuando haya un entorno desplegado, y por eso no se incluyeron en la matriz como criterios que «cumplen».